

CS432: Databases

Introduction to NoSQL Databases

Instructor
Yogesh K. Meena

Lecture No.
TBD

CSE, IIT Gandhinagar
TBD

Lecture Overview

- Challenges of using SQL databases
- Understanding NoSQL databases and MongoDB



Challenges with Relational Databases

- Master-Slave architecture
 - All writes are written to the master. All reads are performed against the replicated slave databases.
 - Good for mostly read, very few update applications.
 - Critical reads may be incorrect as writes may not have been propagated down.
 - Large data sets can pose problems as master needs to duplicate data to slaves

Challenges with Relational Databases

- Partitioning
 - Divide the database across many machines
 - Examples: hash or range partitioning
- Handled transparently by parallel databases
 - But they are expensive
- “Sharding.”
 - Divide data amongst many cheap databases (MySQL/PostgreSQL)
 - Manage parallel access in the application
 - Scales well for both reads and writes
 - Not transparent, application needs to be partition-aware

Why NoSQL Databases?

- Explosion of social media sites (Facebook, Twitter) with large data needs.
- Explosion of storage needs in large websites such as Google, Yahoo
 - Much of the data is not files
- Rise of cloud-based solutions such as Amazon S3 (simple storage solution)
- Shift to dynamically-typed data with frequent schema changes
- Open-source community

Distributed Key-Value Data Sources

- Distributed key-value data storage systems allow key-value pairs to be stored (and retrieved on key) in a massively parallel system
 - E.g., Google BigTable, Yahoo! Sherpa/PNUTS, Amazon Dynamo, ..
- Partitioning, high availability, etc completely transparent to the application
- Sharding systems and key-value stores don't support many relational features
 - No join operations (except within partition).
 - No referential integrity constraints across partitions.

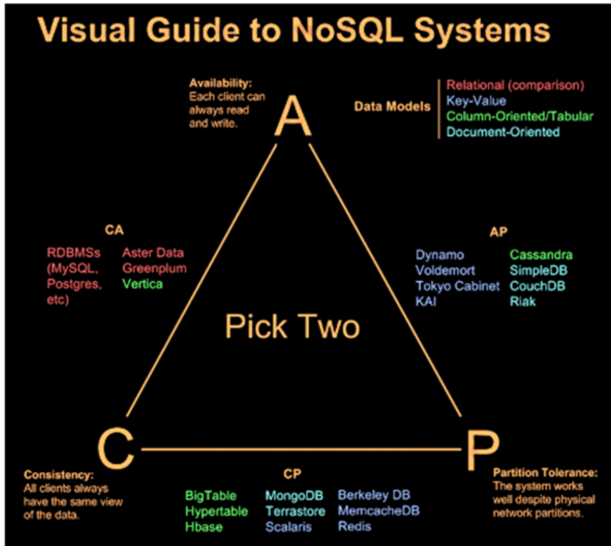
Types of NoSQL Data Storage

Type	Description	Examples
Key-Value Store	Fast key-based access	Redis, DynamoDB
Document Store	JSON-like documents	MongoDB, CouchDB
Column Store	Column-wise analytics	Cassandra, HBase
Graph DB	Relationship-heavy data	Neo4j, ArangoDB

CAP Theorem

- Three properties of a system
 - Consistency (all copies have same value)
 - Availability (system can run even if parts have failed)
 - Partitions (network can break into two or more parts, each with active systems that can't talk to other parts)
- Brewer's CAP "Theorem": You can have at most two of these three properties for any system
- Very large systems will partition at some point
 - Choose one of consistency or availability
 - Traditional databases choose consistency
 - Most Web applications choose availability
 - Except for specific parts such as order processing

CAP Theorem



Availability

- Traditionally, thought of as the server/process available five 9's (99.999 %).
- However, for large node system, at almost any point in time there's a good chance that a node is either down or there is a network disruption among the nodes.
 - Want a system that is resilient in the face of network disruption

Eventual Consistency

- When no updates occur for a long period of time, eventually all updates will propagate through the system and all the nodes will be consistent
- For a given accepted update and a given node, eventually either the update reaches the node or the node is removed from service
- Known as BASE (Basically Available, Soft state, Eventual consistency), as opposed to ACID
 - Soft state: copies of a data item may be inconsistent
 - Eventually Consistent – copies becomes consistent at some later time if there are no more updates to that data item

Common Advantages

- Cheap, easy to implement (open source).
- Data are replicated to multiple nodes (therefore identical and fault-tolerant) and can be partitioned.
 - When data is written, the latest version is on at least one node and then replicated to other nodes.
 - Cheap, easy to implement (open source).
 - No single point of failure.
- Easy to distribute.
- Don't require a schema.

MongoDB Introduction

- Joins
- Group by
 - But PNUTS provides an interesting materialized view approach to joins/aggregation.
- ACID transactions
- SQL
- Integration with applications that are based on SQL

When to Use SQL and When NoSQL?

- NoSQL Data storage systems makes sense for applications that need to deal with very large semi-structured data.
 - Log Analysis.
 - Social Networking Feeds.
- Most of us work on organizational databases, which are not that large and have low update/query rates.
 - Regular relational databases are THE correct solution for such applications.

Demo of MongoDB



MongoDB Introduction

- Popular document-oriented NoSQL Database
- Stores data in JSON like format (BSON)
- Developed for speed, flexibility, and developer Productivity
- Open-source with a powerful cloud offering (MongoDB Atlas)

<https://www.mongodb.com/>

MongoDB Key Concepts

- Database → container of collections
- Collection → group of documents (like a table)
- Document → JSON-like object (like a row)

```
{  
  "name": "Alice",  
  "age": 28,  
  "skills": ["Python", "MongoDB"]  
}
```

MongoDB Features

- Schema-less design
- Powerful query language
- Native replication and sharding
- Integration with popular languages (Python, JavaScript, etc.)
- Easy to deploy in the cloud (MongoDB Atlas)

Basic MongoDB Queries (CRUD) 32

Operation	Method	Example	Description
Create	<code>insert_one()</code>	<code>db.users.insert_one({"name": "Ravi"})</code>	Inserts a single document
	<code>insert_many()</code>	<code>db.users.insert_many([...])</code>	Inserts multiple documents
Read	<code>find()</code>	<code>db.users.find({"age": 25})</code>	Finds all matching documents
	<code>find_one()</code>	<code>db.users.find_one({"name": "Ravi"})</code>	Finds first matching document
Update	<code>update_one()</code>	<code>db.users.update_one({"name": "Ravi"}, {"\$set": {"age": 26}})</code>	Updates one document
	<code>update_many()</code>	<code>db.users.update_many({"age": 25}, {"\$inc": {"age": 1}})</code>	Updates multiple docs
Delete	<code>delete_one()</code>	<code>db.users.delete_one({"name": "Ravi"})</code>	Deletes one document
	<code>delete_many()</code>	<code>db.users.delete_many({"age": 30})</code>	Deletes multiple documents

Common MongoDB Operators

Operator	Use Case	Example
\$eq	Equals	{ "age": { "\$eq": 25 } }
\$gt, \$lt	Greater than, less than	{ "age": { "\$gt": 20, "\$lt": 30 } }
\$in	Match any in array	{ "city": { "\$in": ["Delhi", "Pune"] } }
\$exists	Field existence	{ "email": { "\$exists": true } }
\$and, \$or	Logical AND / OR	{ "\$or": [{ "age": 25 }, { "city": "Delhi" }] }
\$regex	Pattern matching (like SQL LIKE)	{ "name": { "\$regex": "^A" } }

Advanced MongoDB Operators

Feature	Example	Purpose
Projection	<code>db.users.find({}, {"name": 1, "_id": 0})</code>	Only return specific fields
Sort	<code>.sort("age", -1)</code>	Sort by age descending
Limit & Skip	<code>.limit(5).skip(10)</code>	Pagination
Aggregation	<code>db.sales.aggregate([...])</code>	Perform complex data transformations

```
[
  { "$match": { "category": "books" } },
  { "$group": { "_id": "$author", "total": { "$sum": "$price" } } }
]
```

Advanced MongoDB Indexes & Performance

- Indexes: Improve query performance, similar to SQL
 - `db.users.create_index("email", unique=True)`
- Explain plans:
 - `db.users.find("age": 30).explain()`

Acknowledgments/Contributions

- Some of the images utilized in these slides are subject to copyright - Abraham Silberschatz, Henry Korth, and S. Sudarshan. Database System Concepts. 6th Edition, McGraw-Hill Education
- Contributor 2024-25, 2025-26 - Yogesh K. Meena, IIT Gandhinagar